

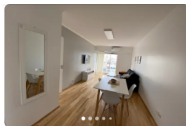
TP2: KNN Y PREDICCIÓN AUTOMÁTICA DE PRECIOS DE ALOJAMIENTOS

MÉTODOS ANALÍTICOS APLICADOS A LOS NEGOCIOS II

Escuela de Negocios, Universidad Torcuato Di Tella

EL PROBLEMA

El problema

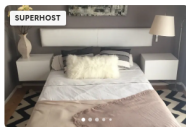


Entire apartment in Recoleta
Depto luminoso en excelente ubicación!!

3 guests · 2 bedrooms · 3 beds · 1 bath
Wifi · Air conditioning · Kitchen

\$29 / night

\$33 total



Entire loft in Recoleta
Tero House ★★★★★

2 guests · 1 bedroom · 1 bed · 1 bath
Wifi · Air conditioning · Kitchen

◆ New lower price

\$13 \$11 / night

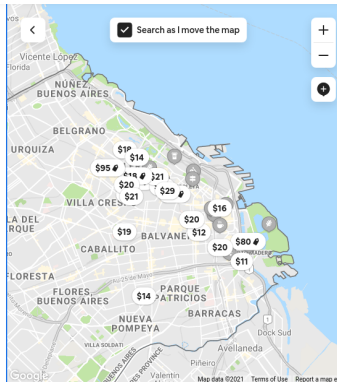
\$21 total

★ 4.78 (137)



Entire apartment in Monserrat
★★★★★Brand New Boutique Balcony Suite i...

2 guests · Studio · 1 bed · 1 bath
Wifi · Pool · Air conditioning



El problema

Lista de registros históricos alojamientos,
con características y precio.



Entire apartment in Recoleta
Depto luminoso en excelente ubicación!

3 guests · 2 bedrooms · 1 bath
Wifi · Air conditioning · Kitchen

★ 4.76 (107)

\$29 / night
[\\$25 total](#)



Entire loft in Recoleta
Tero House ★★★★★

2 guests · 1 bedroom · 1 bath
Wifi · Air conditioning · Kitchen

★ New lower price

★ 4.76 (107)

\$112 \$11 / night
[\\$10 total](#)



Entire apartment in Monserrat
★★★★★ Brand New Boutique Balcony Suite in San ...

2 guests · Studio · 1 bed · 1 bath
Wifi · Pool · Air conditioning

★ Rare find

★ 4.38 (20)

\$20 / night
[\\$18 total](#)



Entire apartment in Palermo
Hermoso dpto en el corazon de palermo

4 guests · 1 bedroom · 4 beds · 1 bath
Wifi · Pool · Air conditioning · Kitchen

★ Rare find

★ 4.68 (92)

\$21 / night
[\\$18 total](#)



Entire apartment in Palermo
Luminoso PH cerca de Plaza Serrano, Palermo Soho

3 guests · 1 bedroom · 2 beds · 1 bath
Wifi · Air conditioning · Kitchen

★ 4.80 (96)

\$20 / night
[\\$18 total](#)

Nuevo departamento, con características
y sin precio.



Entire apartment in Palermo
Strategic location, Equipped and Silent !

2 guests · Studio · 1 bed · 1 bath
Wifi · Air conditioning · Kitchen

★ 4.74 (114)

???? / night
[\\$25 total](#)

Pregunta

A qué precio debería ofrecerse?

El TP

Implementar un método **desde cero** que calcule automáticamente el precio de cada registro aprendiendo de los registros históricos.

LOS DATOS

Los datos: registro histórico

Tenemos un archivo csv con información completa (incluido el precio) de anuncios de airbnb:

- La primera fila es el nombre de las *features*.
- Cada fila es un anuncio distinto definido por sus *features*.

```
"price","latitude","longitude","accommodates","bathrooms","bedrooms","minimum_nights",  
"review_scores_location","review_scores_value"
```

```
40,51.56802,-0.11121,2,1,1,1,21,97,9,9  
75,51.48796,-0.16898,2,1,1,10,89,96,10,9  
307,51.52195,-0.14094,6,2,3,4,42,94,10,9  
29,51.57224,-0.20906,2,1.5,1,10,129,96,9,10  
65,51.46507,-0.32421,2,1,1,1,6,90,10,8  
195,51.47934,-0.28066,5,1.5,3,3,79,96,10,9  
72,51.58461,-0.1617,2,0,1,2,528,97,10,10  
80,51.53972,-0.05885,2,1,1,6,52,97,10,10  
70,51.46871,-0.06455,2,1,1,2,55,96,9,10
```

La descripción de los features es:

- price: costo por noche de alojamiento.
- latitude: coordenada ubicación
- longitude: coordenada ubicación
- accomodates: cantidad máxima de personas habilitadas
- bedrooms: cantidad de habitaciones
- minimum_nights: cantidad mínima de noches para reservar
- number_or_reviews: cantidad de reviews
- review_scores_rating: evaluación general
- review_scores_location: evaluación general de la ubicación
- review_scores_value: evaluación respecto a la relación precio / calidad

En este trabajo, todos los features son numéricos. Nuestra hipótesis es que el precio es una función que depende de las otras características.

Tenemos un archivo csv de registros con sus características pero **sin** el precio:

- La primera fila es el nombre de las *features*.
- Cada fila es un anuncio distinto definido por sus *features*.

latitude,longitude,accommodates,bathrooms,bedrooms,minimum_nights,number_of

51.48756,-0.16915,2,1,1,7,1,100,10,10

51.51458,-0.17249,6,2,3,7,16,93,10,9

- El significado de los features es el mismo.
- Para cada registro: **A qué precio debería anunciarse?**

El TP

Implementar un método **desde cero** que calcule automáticamente el precio de cada registro aprendiendo de los registros históricos.

EL MODELO

Vecinos más cercanos (kNN)

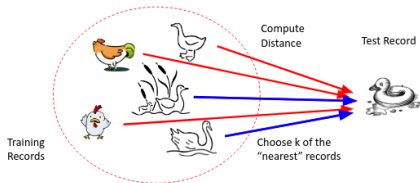
kNN

Es un método *no paramétrico* muy simple usado para problemas de clasificación y regresión en *machine learning*.



Intuición:

- Versión clasificación: *Si camina como pato, grazna como pato, probablemente sea un pato.*
- Versión regresión: *Si camina como pato, grazna como pato, seguro sus características sean las de un pato promedio.*



Vecinos más cercanos (kNN)

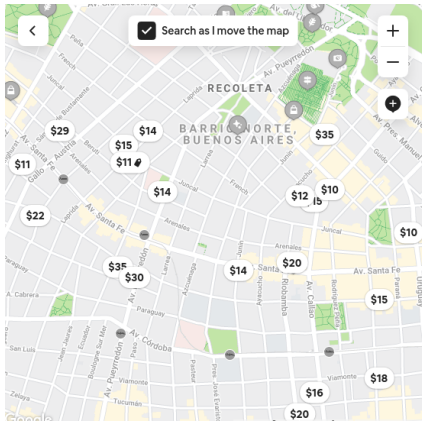
Asumimos que tenemos:

- Un conjunto de registros históricos (training)
- Una medida de distancia entre los datos
- El valor k , la cantidad de vecinos más cercanos a consultar
- Uno o más registros con la variable a predecir

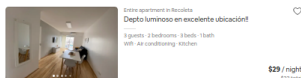
Dado una nueva observación:

- Se calcula la distancia a los registros de training
- Se identifican los k vecinos más cercanos
- Promediamos la variable a predecir de esos vecinos

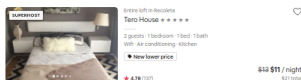
Usar solamente la posición geográfica puede ser muy limitado (diferencia de ambientes, personas, etc.)



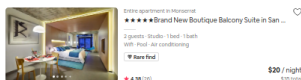
Representamos cada registro de la base de training con un vector n -dimensional con sus atributos.



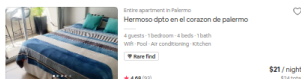
29 (51.56802, -0.11121, 2, 1, 1, 1, 21, 97, 9, 9)



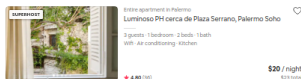
11 (51.48796, -0.16898, 2, 1, 1, 10, 89, 96, 10, 9)



20 (51.52195, -0.14094, 6, 2, 3, 4, 42, 94, 10, 9)



21 (51.57224, -0.20906, 2, 1.5, 1, 10, 129, 96, 9, 10)



20 (51.46507, -0.32421, 2, 1, 1, 1, 6, 90, 10, 8)

Disclaimer: los números de los vectores son indicativos y no se corresponden con la imagen.

Vecinos más cercanos (kNN)

Entra a nuestra plataforma una nueva publicación, con sus características y queremos buscarle precio



Entire apartment in Palermo
Strategic location, Equipped and Silent!
2 guests · Studio · 1 bed · 1 bath
WiFi · Air conditioning · Kitchen

★ 4.34 (114)

???? / night
\$25 total

???? (51.48756, -0.16915, 2, 1, 1, 7, 1, 100, 10, 10)

Supongamos que queremos los $k = 3$ vecinos más cercanos. El método propone:

1. Calcular la **distancia** del nuevo registro a todos los registros de la base de entrenamiento. Vamos a usar la *distancia euclídea*. Dados $x = (x_1, \dots, x_n)$ y $\bar{x} = (\bar{x}_1, \dots, \bar{x}_n)$, la distancia entre x y $\bar{x}$ está dada por

$$\text{dist}(x, \bar{x}) = \sqrt{\sum_{i=1}^n (x_i - \bar{x}_i)^2}$$

2. Elegir los 3 con menor distancia.
3. Tomar como precio el promedio el *precio (price)* de esos tres vecinos.

Observación

Si bien acá estamos trabajando sobre los datos originales, por razones que exceden el contexto de la materia, los datos suelen *preprocesarse* (escalarsse, normalizarse). *Este paso va a existir en el TP, pero ya se lo damos hecho.*

Vecinos más cercanos (kNN)



Entire apartment in Recoleta
Depto luminoso en excelente ubicación!

3 guests · 2 bedrooms · 3 beds · 1 bath
WiFi · Air conditioning · Kitchen

\$29 / night
\$33 total

29 (51.56802, -0.11121, 2, 1, 1, 1, 21, 97, 9, 9)



Entire loft in Recoleta
Toro House *****

2 guests · 1 bedroom · 1 bed · 1 bath
WiFi · Air conditioning · Kitchen

New lower price

\$118 / night
\$121 total

11 (51.48796, -0.16898, 2, 1, 1, 1, 10, 89, 96, 10, 9)



Entire apartment in Monserrat
*******Brand New Boutique Balcony Suite in San ...**

2 guests · Studio · 1 bed · 1 bath
WiFi · Pool · Air conditioning

Rare find

\$20 / night
\$20 total

20 (51.52195, -0.14094, 6, 2, 3, 4, 42, 94, 10, 9)



Entire apartment in Palermo
Hermoso dpto en el corazon de palermo

4 guests · 1 bedroom · 4 beds · 1 bath
WiFi · Pool · Air conditioning · Kitchen

Rare find

\$21 / night
\$21 total

21 (51.57224, -0.20906, 2, 1.5, 1, 1, 10, 129, 96, 9, 10)



Entire apartment in Palermo
Luminoso PPH cerca de Plaza Serrano, Palermo Soho

3 guests · 1 bedroom · 2 beds · 1 bath
WiFi · Air conditioning · Kitchen

\$20 / night
\$20 total

20 (51.46507, -0.32421, 2, 1, 1, 1, 6, 90, 10, 8)

Entonces, el precio para el nuevo BnB es:



Entire apartment in Palermo
Strategic location, Equipped and Silent !

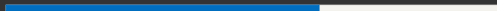
2 guests · Studio · 1 bed · 1 bath
WiFi · Air conditioning · Kitchen

4.36 (114)

???? / night
\$32 total

23 (51.48756, -0.16915, 2, 1, 1, 7, 1, 100, 10, 10)

EL TP



Este trabajo práctico va a estar dividido en cuatro etapas.



- Algunas etapas tienen funcionalidades ya implementadas.
- Se provee un archivo guía con una modularización en funciones para completar.
- Al completar todas las funciones, se tendrán todas las etapas implementadas.
- Una ejecución del programa requiere un archivo csv con la base de training, uno con la base de test, un valor de k y un archivo de salida para escribir las predicciones.

```
input_training = 'training_small.csv'
input_test = 'test_small.csv'
output_pred = 'prediction.csv'
k = 3
```


Procesamiento del input con la base de training.

1. Tenemos el archivo (con los precios)

```
"price","latitude","longitude","accommodates"...  
40,51.56802,-0.11121,2,1,1,1,21,97,9,9  
75,51.48796,-0.16898,2,1,1,10,89,96,10,9  
307,51.52195,-0.14094,6,2,3,4,42,94,10,9  
29,51.57224,-0.20906,2,1.5,1,10,129,96,9,10  
65,51.46507,-0.32421,2,1,1,1,6,90,10,8  
195,51.47934,-0.28066,5,1.5,3,3,79,96,10,9  
72,51.58461,-0.1617,2,0,1,2,528,97,10,10  
80,51.53972,-0.05885,2,1,1,6,52,97,10,10  
70,51.46871,-0.06455,2,1,1,2,55,96,9,10
```

2. Extraemos la info en una lista de listas (funcion `file_to_matrix(filename)`)

```
[[40.0, 51.56802, -0.11121, 2.0, 1.0, 1.0, 1.0, 21.0, 97.0, 9.0, 9.0],  
 [75.0, 51.48796, -0.16898, 2.0, 1.0, 1.0, 10.0, 89.0, 96.0, 10.0, 9.0],  
 [307.0, 51.52195, -0.14094, 6.0, 2.0, 3.0, 4.0, 42.0, 94.0, 10.0, 9.0],  
 [29.0, 51.57224, -0.20906, 2.0, 1.5, 1.0, 10.0, 129.0, 96.0, 9.0, 10.0],  
 [65.0, 51.46507, -0.32421, 2.0, 1.0, 1.0, 1.0, 6.0, 90.0, 10.0, 8.0],  
 [195.0, 51.47934, -0.28066, 5.0, 1.5, 3.0, 3.0, 79.0, 96.0, 10.0, 9.0],  
 [72.0, 51.58461, -0.1617, 2.0, 0.0, 1.0, 2.0, 528.0, 97.0, 10.0, 10.0],  
 [80.0, 51.53972, -0.05885, 2.0, 1.0, 1.0, 6.0, 52.0, 97.0, 10.0, 10.0],  
 [70.0, 51.46871, -0.06455, 2.0, 1.0, 1.0, 2.0, 55.0, 96.0, 9.0, 10.0]]
```

3. Separamos precio (`y_training`) del resto de los datos (`X_training`). Funciones `get_features(dataset)` y `get_y(dataset)`

```
X_training =  
[[51.56802, -0.11121, 2.0, 1.0, 1.0, 1.0, 21.0, 97.0, 9.0, 9.0],  
 [51.48796, -0.16898, 2.0, 1.0, 1.0, 10.0, 89.0, 96.0, 10.0, 9.0],  
 [51.52195, -0.14094, 6.0, 2.0, 3.0, 4.0, 42.0, 94.0, 10.0, 9.0],  
 [51.57224, -0.20906, 2.0, 1.5, 1.0, 10.0, 129.0, 96.0, 9.0, 10.0],  
 [51.46507, -0.32421, 2.0, 1.0, 1.0, 1.0, 6.0, 90.0, 10.0, 8.0],  
 [51.47934, -0.28066, 5.0, 1.5, 3.0, 3.0, 79.0, 96.0, 10.0, 9.0],  
 [51.58461, -0.1617, 2.0, 0.0, 1.0, 2.0, 528.0, 97.0, 10.0, 10.0],  
 [51.53972, -0.05885, 2.0, 1.0, 1.0, 6.0, 52.0, 97.0, 10.0, 10.0],  
 [51.46871, -0.06455, 2.0, 1.0, 1.0, 2.0, 55.0, 96.0, 9.0, 10.0]]  
  
y_training = [40.0, 75.0, 307.0, 29.0, 65.0, 195.0, 72.0, 80.0, 70.0]
```

- Notar que los datos de test (`X_test`) no tienen el precio.
- Simplemente se aplican los pasos 1 y 2.

Estandarizamos los valores de X_{training} y X_{test} . (Mostramos solo X_{test})

```
X_test = [[-0.7459, -0.0028, -0.5276, -0.2156, -0.5345, 0.7921, -0.7281, 2.1549, 0.7071, 0.9999],  
          [-0.1404, -0.0420, 2.1859, 1.7253, 1.8708, 0.7921, -0.6290, -1.1562, 0.7071, -0.5000]]
```

El código ya se encuentra dado en el módulo preprocessing. Solamente es necesario implementar la función media

```
# Estandarizamos los features. Para evitar data leakage, usamos  
    la media  
# y la desviacion estandar solo de los datos de training  
print('Normalizing.... ')  
preproc.standarization(X_training, X_test)
```

Implementamos el algoritmo de kNN que, para cada fila de la matriz X_{test} , realiza:

- calcula la distancia del registro a predecir contra todos los registros de la matriz X_{training} (funciones `euclidean_distance(x,y)` y `get_distances(target, X, y_training)`);
- con las distancias ya calculadas, busca los k vecinos más cercanos (función `get_nearest_neighbors(distancias, k)`);
- retorna el promedio de los precios de los k vecinos más cercanos (función `predict(X_training, y_training, new_obs, k)`, y llama a la de los puntos anteriores)

La función `knn` es la que recorre todas las observaciones a predecir, llama a `predict` y guarda el valor predicho en una lista. Ejemplo de lo que devuelve `knn`:

`X_training, y_training, new_obs, k`

Finalmente, escribimos el vector de predicciones en un archivo (especificado por la variable `output_pred`) con el header en la primera línea, y a continuación en cada línea el número de fila (numerando desde cero) y la predicción correspondiente.

```
id,predicted
```

```
0,75.0
```

```
1,192.3333333333334
```

Sobre los datos

Tenemos dos bases de training / test:

- base small: 10 registros de training, 2 registros a predecir (test)
- base estándar: 48289 registros de training, 1000 registros a predecir. Para esta base **no** van a conocer el valor real, y van a subir el archivo de output a la plataforma *kaggle* donde se evaluará la calidad.

Pueden ejecutar variando el valor de k a fin de mejorar la métrica de evaluación.

MÁS ALLÁ DEL TP

... que no contempla el TP:

- cómo hacemos para evaluar cuál es el valor de k que conviene tomar?
- los resultados que tenemos se mantienen se cambiamos la base de test?
- estamos haciendo trampa si probamos múltiples veces contra *kaggle* optimizando (minimizando) la métrica de evaluación?
- *Over-qué? Overfitting?*

Nosotros estamos tomando el algoritmo como excusa y caso de uso para practicar programación.

Todas estas preguntas (y muchas más) se responden en Introducción a Data Science!